

Full Stack Web Development

Topics: Laravel Web Development from Scratch

Instructor

Amitosh Gain

Manager(Software Development)

Institute of Information & Communication Technology (IICT)

Dhaka University of Engineering & Technology (DUET)

LARAVEL 12 — BEGINNERS

Personal Task Board from Scratch

MVC · Routing · Database · Auth · Blade · CRUD

Theory in class · Practice hands-on



Personal Task Board

A simple to-do / task manager where a logged-in user can create, view, edit, and delete their own tasks. Each task has a title, description, and a status (pending / done).



Register & Log In

Only you can see your own tasks.



Create Tasks

Add a title, description, and pick a status.



Edit & Delete

Update or remove tasks you created.



Task List

See all your tasks on one clean page.

6 Steps to a Working App

01

Setup & Project Structure

Install Laravel, understand folders

02

Routing & Controllers

Map URLs to PHP code

03

Database & Migrations

Design the tasks table

04

Eloquent Models

Read and write tasks with PHP

05

Authentication

Register, login, logout

06

CRUD & Blade Views

Full create/edit/delete with forms

How a Laravel App is Organised

app/

All PHP logic lives here — Models, Controllers, and more.

resources/views/

Blade templates — the HTML pages your users see.

routes/web.php

Maps every URL to a controller action.

database/migrations/

PHP files that create/change database tables.

Browser

→ Route

→ Controller

→ Model/DB

→ Blade View

How Requests Reach Your Code

Routes (web.php)

GET	/tasks	Show all tasks
GET	/tasks/create	Show create form
POST	/tasks	Save new task
GET	/tasks/{task}/edit	Show edit form
PUT	/tasks/{task}	Update the task
DELETE	/tasks/{task}	Delete the task

TaskController.php

<code>index()</code>	Load all tasks, pass to view
<code>create()</code>	Return the create form view
<code>store()</code>	Validate input, save task
<code>edit()</code>	Load one task, show edit form
<code>update()</code>	Validate input, update task
<code>destroy()</code>	Delete task, redirect back

Designing the Tasks Table

What are Migrations?

- A migration is a PHP file that creates a database table.
- You run `php artisan migrate` to apply all migrations.
- You can roll back with `php artisan migrate:rollback`.
- Never edit old migration files — add a new one instead.
- This way your database changes are version-controlled, just like code.

tasks table schema

id	<code>bigInteger</code>	Auto primary key
user_id	<code>foreignId</code>	Links to users table
title	<code>string</code>	Required, max 255
description	<code>text</code>	Optional longer text
status	<code>enum</code>	'pending' or 'done'
timestamps	—	created_at, updated_at

Reading & Writing Tasks with PHP

Eloquent is Laravel's ORM. It maps each database table to a PHP class. No raw SQL needed.

Get all tasks

```
Task::all()
```

Get one task

```
Task::find($id)
```

Create a task

```
Task::create([...])
```

Delete a task

```
$task->delete()
```

Relationship: A User hasMany Tasks · A Task belongsTo a User

Register, Login & Protect Your Pages

1

Register

User fills in name, email, password. Laravel hashes the password and saves them.

2

Login

Laravel checks email + password against DB. On match, starts a session.

3

Session

A cookie keeps the user logged in. `Auth::user()` gives you their data anywhere.

4

Middleware

->`middleware('auth')` on a route blocks guests from seeing protected pages.

Tip: Laravel Breeze installs ready-made register/login pages with one command: `php artisan breeze:install`

Building the UI with Blade Templates

Blade is Laravel's template engine. It lets you mix PHP logic inside your HTML pages cleanly.

tasks/index.blade.php

Task List Page

Loop through all tasks and display them in a table. Show Edit and Delete buttons.

```
@foreach($tasks as $task)
```

tasks/create.blade.php

Create Task Form

An HTML form with title, description, status dropdown. Posts to POST /tasks.

```
<form method='POST'  
action='/tasks'>
```

tasks/edit.blade.php

Edit Task Form

Same form pre-filled with the task's current data. Posts to PUT /tasks/{id}.

```
@method('PUT')
```

How Create, Read, Update, Delete Works

C — Create

- User opens `/tasks/create`
- Fills in the form
- Submits → `POST /tasks`
- `store()` validates & saves
- Redirects to task list

R — Read

- User opens `/tasks`
- `index()` calls `Task::all()`
- Passes `$tasks` to Blade
- Blade loops & shows them

U — Update

- User clicks Edit
- `edit()` loads the task
- User changes fields
- Submits → `PUT /tasks/{id}`
- `update()` saves changes

D — Delete

- User clicks Delete button
- Sends DELETE request
- `destroy()` finds task
- `$task->delete()`
- Redirects back to list

Never Trust User Input

Before saving anything to the database, always validate the data the user submitted.

In your store() method:

```
$validated = $request->validate([  
    'title' => 'required|max:255',  
    'description' => 'nullable|string',  
    'status' => 'required|in:pending,done',  
]);
```

```
Task::create($validated);  
return redirect('/tasks');
```

What happens?



Valid data

Laravel saves to DB and redirects.



Invalid data

Laravel sends back errors automatically.



Error display

Blade shows errors with @error tag.

Write Once, Use Everywhere

✗ Copy-pasting the same form field

```
<!-- In create.blade.php -->
<div>
    <label>Title</label>
    <input name='title' type='text'>
</div>

<!-- In edit.blade.php -->
<div> <!-- SAME CODE AGAIN -->
    <label>Title</label>
    <input name='title' type='text'>
</div>
```

✓ One component, used anywhere

```
<!-- Define once in -->
<!-- form-input.blade.php -->
<div>
    <label>{{ $label }}</label>
    <input name='{{ $name }}'
        type='text'>
</div>

<!-- Use anywhere -->
<x-form-input name='title'
    label='Title' />
```

Generate Fake Data for Development

Instead of manually creating tasks to test your app, use Factories to generate them instantly.

1

Factory

TaskFactory.php defines fake data rules using Faker library.

```
'title' => fake()->sentence(4)
```

2

Seeder

TaskSeeder.php calls the factory to create many records at once.

```
Task::factory(20)->create()
```

3

Run it

One artisan command fills your database with 20 test tasks instantly.

```
php artisan db:seed
```

You Built a Real Laravel App! 🎉

What you learned:

- Laravel project structure
- Routing & resource controllers
- Database migrations & schema design
- Eloquent models & relationships
- User authentication with Breeze
- Blade templates & components
- Full CRUD (create/read/update/delete)
- Form validation
- Factories & database seeding